

Automatización del Etiquetado de Productos mediante Transformadores de Visión

Visión por Computadora III

Carrera de Especialización en Inteligencia Artificial

Autores:

Abril Noguera
Pedro Lucas Barrera
Lautaro Gabriel Medina

Ciudad de Buenos Aires, diciembre de 2025

Resumen

El proyecto desarrollado consiste en la construcción de un sistema capaz de identificar y asignar atributos descriptivos a productos a partir de sus imágenes mediante un modelo de aprendizaje profundo basado en transformadores de visión. El sistema aborda una necesidad concreta del comercio digital, donde el etiquetado manual de artículos representa una tarea costosa, lenta y con alta probabilidad de errores. La solución propuesta automatiza este proceso, mejora la calidad del catálogo y permite acelerar la incorporación de nuevos productos en plataformas de venta en línea.

Índice general

Resumen	I
1. Introducción general	1
1.1. Contexto y motivación	1
1.2. Caso de negocio	1
1.3. Propuesta de valor	2
1.4. Objetivos y alcance	2
1.5. Datos utilizados	3
2. Análisis Exploratorio de Datos	5
2.1. Descripción general del dataset	5
2.2. Análisis de valores faltantes	5
2.3. Distribución de clases	6
2.4. Inspección visual de muestras	8
2.5. Variabilidad en tamaños de imagen	9
2.6. Análisis de calidad de imagen	9
2.7. Conclusiones del EDA y su impacto en el modelado	10
3. Preparación y Preprocesamiento	13
3.1. División del dataset	13
3.2. Preprocesamiento de imágenes	14
3.3. Data augmentation	14
3.4. Construcción del PyTorch Dataset	15
3.5. Selección de modelo	15
3.5.1. Descripción de la arquitectura ViT-B/16	15
3.5.2. Criterios de selección	16
3.6. Configuración de entrenamiento	16
3.6.1. Función de pérdida	17
3.6.2. Optimizador y regularización	17
3.6.3. Scheduler de <i>learning rate</i>	17
3.6.4. Número de épocas, <i>batch size</i> y <i>early stopping</i>	17
3.6.5. Data augmentation durante el entrenamiento	18
4. Resultados y Conclusiones	19
4.1. Resultados	19
4.2. Conclusiones	20
4.2.1. Entrenamiento y validación	20
4.2.2. Análisis de mapas de atención	21
4.2.3. Limitaciones y recomendaciones finales	22
A. Planificación del Proyecto	23

Índice de figuras

2.1. Proporción de valores faltantes en el archivo <code>styles.csv</code>	6
2.2. Distribución de clases para <code>articleType</code>	6
2.3. Distribución de clases para <code>subCategory</code>	7
2.4. Distribución de clases para <code>masterCategory</code>	7
2.5. Distribución de clases para <code>gender</code>	8
2.6. Muestras aleatorias de imágenes del dataset.	8
2.7. Análisis de brillo, contraste, nitidez y distribución de color.	10
4.1. Evolución de <i>train loss</i> y <i>validation loss</i> para los distintos experimentos.	20
4.2. Comparación del <i>validation loss</i> entre los experimentos evaluados.	21
4.3. Ejemplo de mapa de atención generado por el modelo.	22

Índice de tablas

4.1. Métricas de desempeño del modelo en el conjunto de <i>test</i>	19
A.1. Planificación del Proyecto	23

Capítulo 1

Introducción general

El presente capítulo introduce el contexto general del proyecto, expone el problema que motivó su desarrollo, presenta el caso de negocio y describe los datos utilizados. Asimismo, se detallan los objetivos del trabajo y su alcance, estableciendo el marco conceptual necesario para comprender las decisiones técnicas y metodológicas que se desarrollan en los capítulos posteriores.

1.1. Contexto y motivación

El crecimiento del comercio electrónico y la disponibilidad masiva de catálogos digitales han incrementado la necesidad de organizar y clasificar grandes volúmenes de imágenes de productos. En este escenario, la automatización del etiquetado visual se ha convertido en una herramienta fundamental para mejorar la eficiencia operativa y garantizar la consistencia en la gestión de catálogos.

El proyecto desarrollado surgió a partir de esta necesidad: se implementó un sistema de etiquetado automático de productos basado en modelos de visión por computadora. Dicho sistema permitió reducir la dependencia del etiquetado manual, disminuir errores humanos y acelerar el procesamiento de catálogos. Esta iniciativa se orientó a evaluar la utilidad de arquitecturas modernas como los *Vision Transformers* en problemas de clasificación multi-etiqueta.

1.2. Caso de negocio

El crecimiento acelerado del comercio electrónico ha generado catálogos con miles de productos que requieren actualización constante. Entre las tareas fundamentales para mantener dichos catálogos se encuentra la correcta clasificación del tipo de artículo, información que determina cómo se organiza la oferta, cómo se indexan los productos y cómo se distribuyen en categorías de navegación. Este proceso, aunque aparentemente simple, suele realizarse de manera manual en muchas organizaciones.

El etiquetado manual presenta varias limitaciones operativas. Por un lado, implica costos significativos en horas-hombre, especialmente en contextos donde ingresan grandes volúmenes de productos nuevos cada semana. Por otro lado, el proceso es inherentemente subjetivo: diferentes operadores pueden asignar tipos de artículos distintos a productos similares, generando inconsistencias que afectan la estructura del catálogo. Estas inconsistencias impactan negativamente

en la experiencia del usuario, dificultan la búsqueda y navegación, y pueden incluso comprometer sistemas internos que dependen de categorías confiables, como motores de recomendación o módulos de analítica.

Asimismo, la necesidad de mantener el catálogo actualizado introduce cuellos de botella cuando el volumen de imágenes supera la capacidad del equipo encargado del etiquetado. Esto retrasa la publicación de nuevos productos y limita la agilidad comercial de la plataforma.

En este contexto, se vuelve necesario un sistema automático, confiable y escalable que permita clasificar de manera precisa el tipo de artículo a partir de una imagen. El presente proyecto se enmarca en esta problemática y evalúa la capacidad de modelos basados en *Vision Transformers* para automatizar la clasificación de *articleType* a partir de imágenes de productos de moda.

1.3. Propuesta de valor

El sistema desarrollado automatiza la clasificación del tipo de artículo utilizando un modelo de visión por computadora basado en *Vision Transformers*. Aunque la tarea se concentra exclusivamente en este atributo, su correcta automatización aporta valor significativo al proceso de gestión de catálogos:

- **Reducción de costos operativos:** Elimina gran parte del trabajo manual asociado al etiquetado inicial de productos, disminuyendo el uso de recursos humanos en tareas repetitivas.
- **Consistencia en la categorización:** El modelo aplica criterios uniformes, evitando la variabilidad entre operadores y garantizando una estructura de catálogo más coherente.
- **Agilidad en la carga de productos:** La clasificación automática permite acelerar el flujo de incorporación de nuevos artículos, reduciendo el tiempo entre la disponibilidad del producto y su publicación efectiva.
- **Mejor organización del catálogo:** Una clasificación precisa de *articleType* mejora la navegación, la indexación y la calidad de los filtros disponibles para los usuarios.
- **Escalabilidad operativa:** El sistema puede procesar miles de imágenes de manera eficiente sin requerir aumentos proporcionales en la dotación de personal.

Así, la propuesta de valor se centra en un componente puntual pero fundamental del catálogo: la clasificación del tipo de artículo. Automatizar esta etapa contribuye directamente a la eficiencia operativa, la calidad del contenido y la experiencia del usuario, alineándose con las necesidades de plataformas de comercio electrónico modernas.

1.4. Objetivos y alcance

El proyecto tuvo como objetivo general desarrollar un sistema capaz de etiquetar automáticamente imágenes de productos de moda utilizando modelos basados en *Vision Transformers*. Este sistema buscó reproducir y estandarizar el proceso de

etiquetado que habitualmente se realiza de manera manual, evaluando su capacidad para predecir atributos clave del catálogo tales como categoría, tipo, color y género del producto.

En términos más específicos, el trabajo se propuso:

- Construir un pipeline reproducible de procesamiento de datos que incluyera la descarga, validación y limpieza del dataset original.
- Implementar un modelo de clasificación multi-etiqueta basado en *Vision Transformers*, adaptado a las características del dataset.
- Entrenar y validar el modelo mediante particiones estratificadas, asegurando una evaluación equilibrada de las distintas clases.
- Analizar el desempeño del modelo utilizando métricas estandarizadas como *accuracy*, *F1*, *precision* y *recall*.
- Examinar los errores y sesgos presentes en las predicciones, identificando limitaciones del enfoque.
- Desarrollar una herramienta de inferencia que permitiera aplicar el modelo a nuevas imágenes en un entorno práctico.

El alcance del proyecto se limitó al análisis y desarrollo de un prototipo funcional entrenado sobre un dataset público. No se abordaron aspectos propios de un sistema productivo, tales como el entrenamiento continuo, la integración con plataformas de comercio electrónico, la optimización de tiempos de inferencia para altos volúmenes ni la incorporación de retroalimentación humana en el ciclo de etiquetado. Tampoco se realizaron experimentos con arquitecturas alternativas ni con técnicas de aumento extensivo de datos debido a restricciones de tiempo y recursos computacionales.

A pesar de estas limitaciones, el desarrollo realizado permite demostrar la viabilidad de automatizar el etiquetado de productos mediante modelos de visión por computadora modernos, sentando las bases para futuras mejoras orientadas a escenarios reales de operación.

1.5. Datos utilizados

Para el desarrollo del sistema se utilizó el dataset público *Fashion Product Images (Small)*, disponible en Kaggle¹. Este conjunto incluye más de 44.000 imágenes de productos de moda junto con un archivo tabular `styles.csv` que contiene información estructurada del catálogo.

El dataset original presenta atributos como categoría general del producto, subcategoría, tipo, género, temporada y color dominante. Cada imagen está identificada mediante un `productId`, lo que permite vincularla con sus metadatos tabulares. Antes de poder utilizarse en el pipeline de entrenamiento, se aplicaron procesos de limpieza, normalización y verificación de integridad debido a la presencia de valores faltantes, clases poco representadas y rutas inconsistentes.

A partir de este conjunto inicial, se construyó un dataset final que incluyó: (i) imágenes validadas y preprocesadas, (ii) atributos seleccionados para la predicción y

¹<https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small>

(iii) una partición estratificada en *train*, *validation* y *test*. Esta estructura permitió evaluar de manera rigurosa el desempeño del modelo entrenado bajo condiciones reales.

Capítulo 2

Análisis Exploratorio de Datos

El presente capítulo describe el análisis exploratorio (EDA) realizado sobre el conjunto de datos utilizado para el entrenamiento del sistema de etiquetado automático de productos. El objetivo del EDA es comprender la estructura del dataset, evaluar la calidad de las imágenes, identificar patrones relevantes y detectar posibles problemas que condicionen las decisiones de preprocesamiento y modelado. A partir de este análisis se establecen los requerimientos y transformaciones necesarias para garantizar un entrenamiento adecuado del modelo basado en *Vision Transformers*.

2.1. Descripción general del dataset

El proyecto emplea el dataset público *Fashion Product Images (Small)*, disponible en Kaggle, que contiene más de 44.000 imágenes de productos de moda junto con metadatos tabulares provistos en el archivo `styles.csv`. Cada imagen está asociada a un identificador único (`id`) y cuenta con atributos descriptivos como:

- **masterCategory**: categoría general del artículo.
- **subCategory**: categoría específica intermedia.
- **articleType**: tipo concreto de producto.
- **gender**: público objetivo.
- **baseColour**, **season**, **usage**, entre otros.

Durante el EDA se realizó una verificación de integridad del archivo tabular y una validación de la existencia y apertura de cada imagen asociada.

2.2. Análisis de valores faltantes

La figura 2.1 muestra la proporción de valores faltantes por atributo. Las variables con mayor incompletitud corresponden a `usage`, `season` y `baseColour`, ninguna de las cuales forma parte de los atributos objetivo del proyecto. Los campos esenciales para la clasificación (`gender`, `masterCategory`, `subCategory` y `articleType`) no presentan valores faltantes.

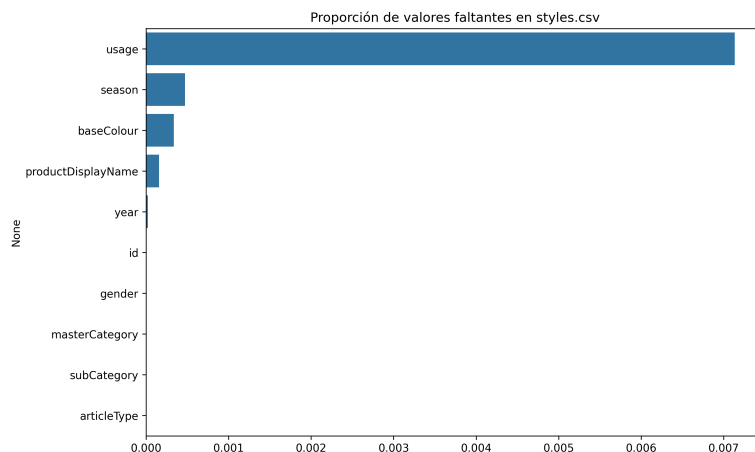


FIGURA 2.1. Proporción de valores faltantes en el archivo `styles.csv`.

Conclusión: No es necesario realizar imputación compleja; el dataset es adecuado para construir las etiquetas objetivo sin modificaciones adicionales.

2.3. Distribución de clases

La figura 2.2 ilustra la distribución de clases para `articleType`. Se observa un marcado desbalance, con categorías como *Tshirts*, *Shirts* y *Casual Shoes* concentrando una cantidad significativamente mayor de muestras respecto de categorías minoritarias.

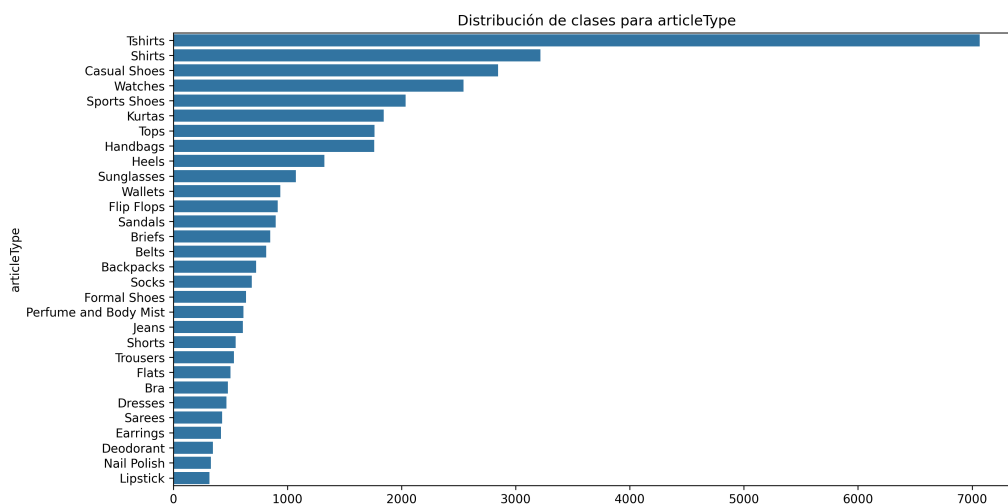


FIGURA 2.2. Distribución de clases para `articleType`.

Patrones similares se observan en `subCategory` y `masterCategory` (figuras 2.3 y 2.4), donde predominan *Topwear*, *Shoes* y *Apparel*, respectivamente.

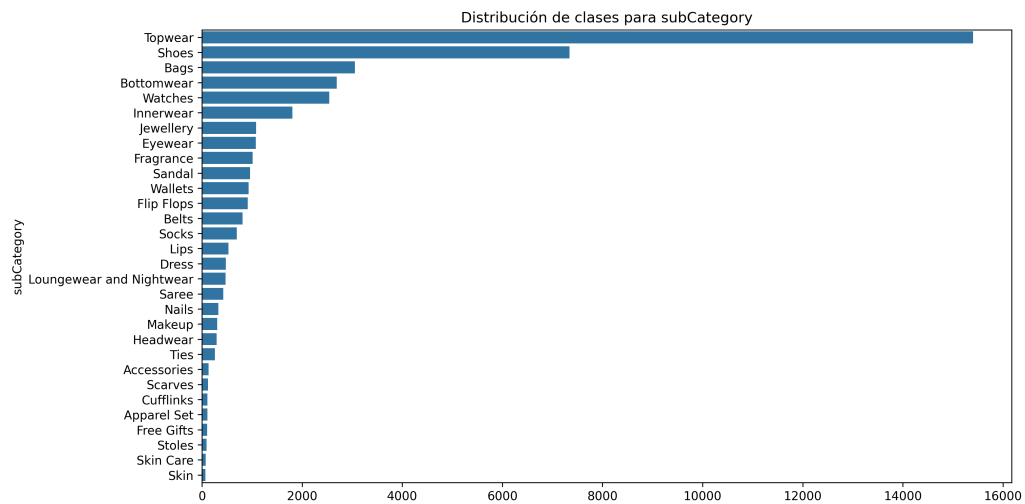


FIGURA 2.3. Distribución de clases para subCategory.

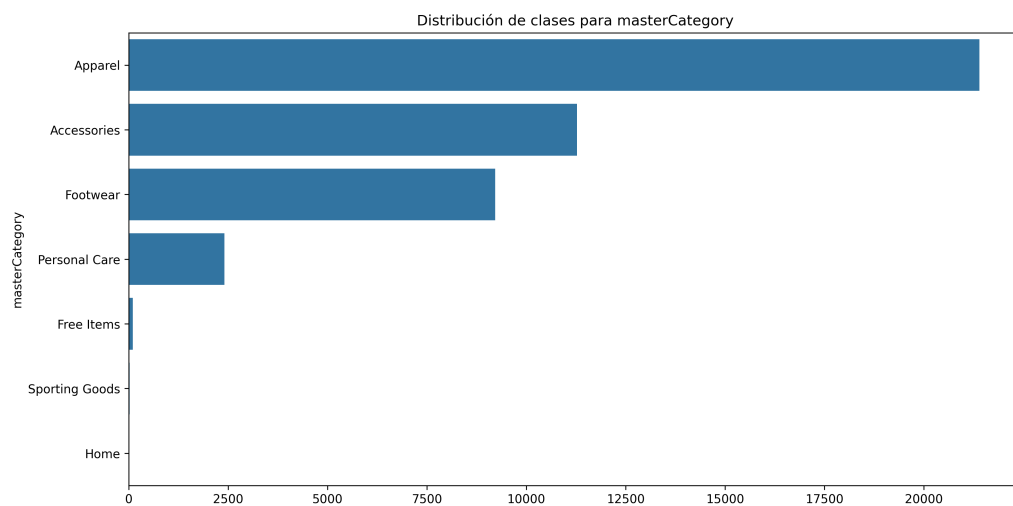


FIGURA 2.4. Distribución de clases para masterCategory.

En cuanto al atributo *gender*, la distribución está dominada por *Men* y *Women*, mientras que *Boys*, *Girls* y *Unisex* representan una proporción menor (figura 2.5).

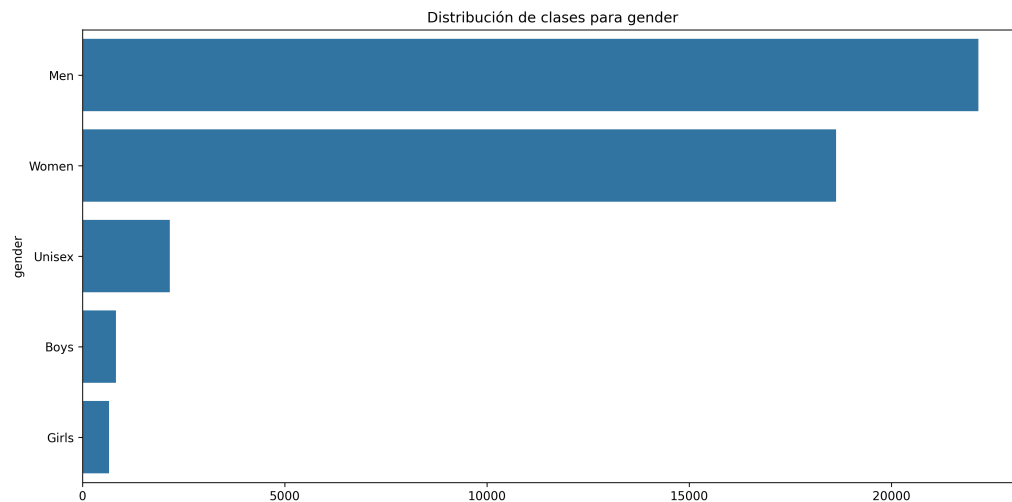


FIGURA 2.5. Distribución de clases para gender.

Conclusión: El dataset presenta un fuerte desbalance de clases que puede afectar el aprendizaje del modelo. Esto justifica la aplicación posterior de estrategias como *class weighting*, muestreo estratificado o técnicas de reponderación durante el entrenamiento.

2.4. Inspección visual de muestras

La figura 2.6 muestra una selección aleatoria de imágenes del dataset. Se observa que la mayoría de los productos están fotografiados sobre fondo blanco, correctamente centrados y con iluminación homogénea. La ausencia de fondos complejos y ruido visual simplifica el aprendizaje del modelo.

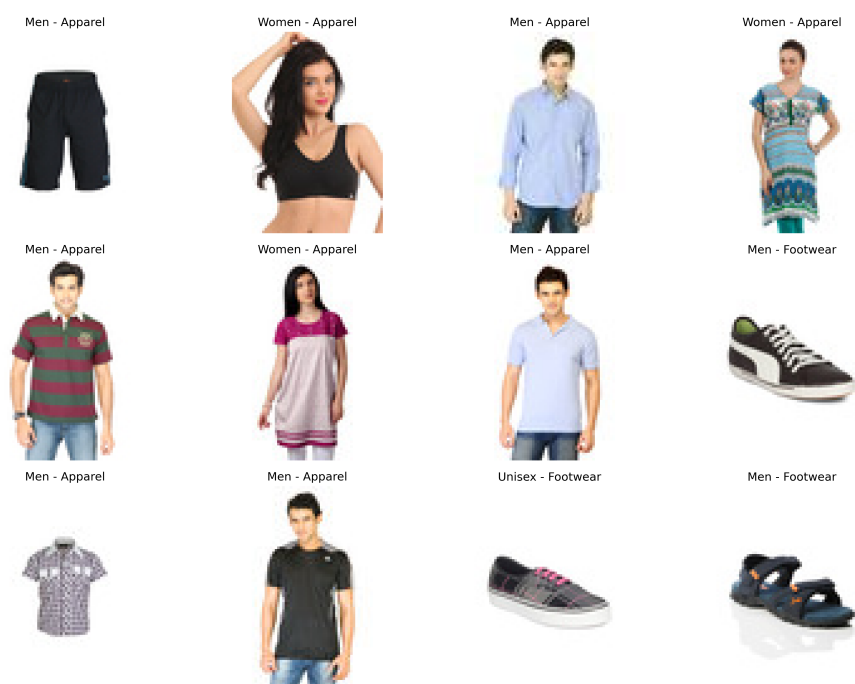


FIGURA 2.6. Muestras aleatorias de imágenes del dataset.

Conclusión: Las condiciones controladas de captura favorecen un desempeño estable en modelos de visión profunda.

2.5. Variabilidad en tamaños de imagen

El análisis de resolución reveló que todas las imágenes tienen un tamaño uniforme de **60×80 px**, con desviación estándar nula. Si bien esta consistencia elimina la necesidad de correcciones geométricas, la resolución es insuficiente para arquitecturas modernas como los *Vision Transformers*, que operan con tamaños estándar de 224×224 px.

Conclusión: Es necesario reescalar todas las imágenes a 224×224 px para compatibilidad con el modelo seleccionado.

2.6. Análisis de calidad de imagen

Se evaluaron métricas complementarias de calidad:

- **Brillo:** valores centrados entre 200 y 230, indicando imágenes bien iluminadas.
- **Contraste:** niveles moderados, consistentes entre productos.
- **Nitidez:** varianza Laplaciana mayormente entre 500 y 4000, suficiente para distinguir bordes.
- **Distribución RGB:** canales balanceados sin dominancia cromática.

Estas mediciones se ilustran en la figura 2.7.

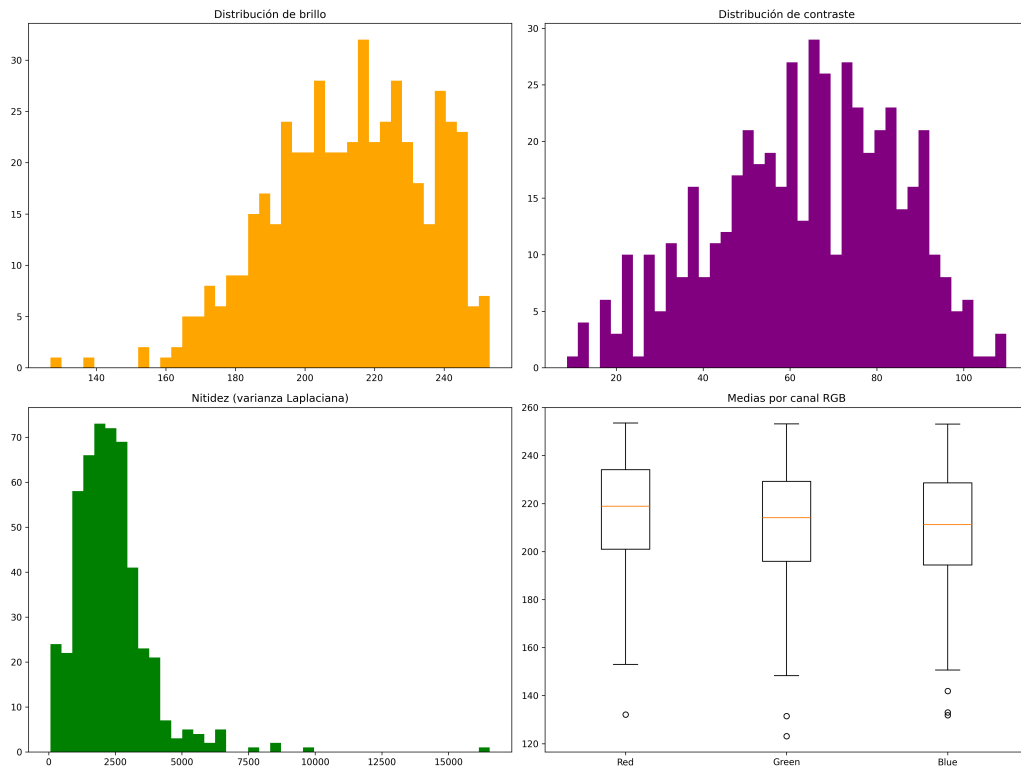


FIGURA 2.7. Análisis de brillo, contraste, nitidez y distribución de color.

Conclusión: La calidad general del dataset es elevada y consistente, lo cual minimiza la necesidad de correcciones avanzadas o normalizaciones fuera del estándar habitual de redes profundas.

2.7. Conclusiones del EDA y su impacto en el modelado

A partir del análisis realizado, se identifican las siguientes decisiones críticas para el pipeline de modelado:

- **Redimensionamiento obligatorio:** todas las imágenes deben ser escaladas a 224×224 px para ser compatibles con *Vision Transformers*.
- **Normalización estándar:** dada la estabilidad de brillo y color, basta con aplicar la normalización típica de modelos preentrenados (mean/std de ImageNet).
- **Augmentations moderados:** la homogeneidad del dataset sugiere utilizar aumentos suaves (flips, jitter ligero) para mejorar generalización sin distorsionar objetos.
- **Manejo de desbalance:** será necesario emplear *class weighting*, muestreo estratificado o pérdidas alternativas para evitar que el modelo favorezca categorías dominantes.
- **Selección del modelo:** la baja resolución original y la estructura limpia de producto sobre fondo blanco justifican el uso de *Vision Transformers*, que

aprovechan patrones espaciales uniformes y generalizan bien ante variaciones mínimas.

En conjunto, el EDA confirma que el dataset es adecuado para entrenar un sistema de clasificación de atributos visuales basado en arquitecturas modernas de visión profunda, requiriendo únicamente ajustes controlados para alcanzar condiciones óptimas de entrenamiento.

Capítulo 3

Preparación y Preprocesamiento

Este capítulo describe el proceso de preparación de datos realizado antes del entrenamiento del modelo. Se detalla cómo se construyeron los conjuntos de *train*, *validation* y *test*, los pasos de preprocesamiento aplicados a las imágenes y las técnicas de *data augmentation* utilizadas para mejorar la capacidad de generalización del modelo. Asimismo, se justifica la necesidad de normalizar y estandarizar las entradas de acuerdo con los requerimientos de la arquitectura *Vision Transformer* seleccionada.

3.1. División del dataset

A partir del conjunto limpio generado en la etapa de análisis exploratorio, se definió una división estratificada en tres subconjuntos: entrenamiento, validación y prueba. La estratificación se realizó tomando como variable objetivo el atributo *articleType*, con el fin de preservar la proporción relativa de clases en cada partición y evitar sesgos durante el entrenamiento.

El dataset final se dividió con los siguientes porcentajes:

- 70 % para entrenamiento.
- 15 % para validación.
- 15 % para prueba.

La división se realizó mediante una estrategia robusta para manejar clases con baja frecuencia. Aquellos tipos de producto con menos de cinco muestras fueron agrupados en una clase adicional *RARE_CLASS* con el fin de evitar errores de estratificación. Esta técnica permitió mantener la coherencia estadística de los subconjuntos sin perder información relevante.

Además de los archivos `train.csv`, `val.csv` y `test.csv`, se generaron carpetas independientes con las imágenes correspondientes a cada partición, asegurando un pipeline reproducible y coherente con las buenas prácticas de *cookiecutter data science*:

```
data/  
  processed/  
    images/  
      train/  
      val/  
      test/
```

Esta organización facilita la carga del dataset en PyTorch, reduce errores de lectura y estandariza el flujo de experimentación.

3.2. Preprocesamiento de imágenes

El preprocesamiento se orientó a adaptar las imágenes del dataset a los requerimientos del modelo *Vision Transformer*, cuya arquitectura espera entradas cuadradas y normalizadas. A partir del análisis realizado en el capítulo de EDA, se observaron las siguientes características relevantes:

- Todas las imágenes poseen tamaño uniforme de **60×80 píxeles**, inferior al requerido por ViT.
- La relación de aspecto es constante, lo que facilita el escalado sin distorsión.
- La calidad de las imágenes (luminosidad, contraste, nivel de enfoque y saturación) es adecuada para un modelo de clasificación supervisada, aunque con variabilidad suficiente como para beneficiar el uso de técnicas de *augmentation*.

En consecuencia, se aplicaron los siguientes pasos de preprocesamiento:

1. **Redimensionamiento a 224×224 píxeles.** Este tamaño es estándar para modelos basados en Transformers y permite aprovechar pesos preentrenados en ImageNet.
2. **Conversión a tensor y normalización.** Se aplicaron las estadísticas de ImageNet:

$$\mu = (0,485, 0,456, 0,406), \quad \sigma = (0,229, 0,224, 0,225)$$

con el fin de estabilizar el entrenamiento y lograr compatibilidad con pesos preentrenados.

3. **Validación de integridad.** Se eliminaron imágenes corruptas o no legibles y se depuraron filas del CSV que no podían vincularse a su archivo correspondiente.

Este pipeline garantiza que todas las imágenes de entrada cumplen el formato necesario para maximizar el desempeño del modelo.

3.3. Data augmentation

Dado que las imágenes del dataset son relativamente pequeñas y presentan poca diversidad visual en términos de iluminación, pose y fondo, se definió un conjunto de transformaciones destinadas a aumentar la variabilidad sintética del conjunto de entrenamiento. Esto reduce el riesgo de sobreajuste y mejora la robustez del modelo.

Las transformaciones aplicadas durante el entrenamiento fueron:

- **Random Horizontal Flip:** simula variaciones naturales en pose y orientación.
- **Color Jitter (brillo, contraste, saturación):** incrementa la tolerancia del modelo a condiciones de iluminación heterogéneas.

- **Random Rotation ($\pm 15^\circ$):** introduce pequeñas perturbaciones para mejorar la invariancia rotacional.
- **Affine transformations:** ligeras modificaciones de escala o traslación, cuando aplicable.

Estas transformaciones se aplican de manera estocástica únicamente al conjunto de entrenamiento. El conjunto de validación y prueba solo recibe las transformaciones estrictamente necesarias (redimensionamiento y normalización), con el objetivo de realizar una evaluación justa y reproducible del modelo.

3.4. Construcción del PyTorch Dataset

Con las particiones y el preprocesamiento definidos, se implementó una clase `PyTorch Dataset` capaz de:

- Cargar imágenes desde las carpetas correspondientes a cada split.
- Aplicar automáticamente las transformaciones definidas para cada partición.
- Mapear cada imagen hacia sus atributos objetivos codificados.
- Soportar clasificación multi-etiqueta mediante vectores binarios por atributo.

Este diseño permite integrar el dataset de manera directa con el *DataLoader* de PyTorch, facilitando el batching, el shuffle y la paralelización en CPU.

3.5. Selección de modelo

Para la tarea de clasificación multiclase sobre imágenes de productos de e-commerce se seleccionó como modelo base un *Vision Transformer* de tipo ViT-B/16, en su implementación de `torchvision` con pesos preentrenados en ImageNet. Esta variante (Base, con tamaño de parche 16×16) se adaptó al problema reemplazando la *classification head* original por una capa totalmente conectada ajustada al número de clases del dataset, manteniendo el resto de la arquitectura estándar y realizando *fine-tuning* sobre el dominio objetivo.

3.5.1. Descripción de la arquitectura ViT-B/16

ViT-B/16 sigue la arquitectura de los *Vision Transformers* clásicos: la imagen de entrada (224×224) se divide en parches no solapados de 16×16 píxeles, que se aplanan y proyectan linealmente a vectores de *embedding*. A esta secuencia se añade un token especial de clasificación (`[CLS]`) y se suman *positional embeddings* entrenables.

La secuencia resultante se procesa mediante una pila de bloques `Transformer encoder`, cada uno compuesto por una capa de *Multi-Head Self-Attention* (MHSA), una red *feed-forward* por token y mecanismos de normalización y residuales. Finalmente, la representación del token `[CLS]` se pasa a la *classification head*, una capa lineal ajustada al número de clases.

3.5.2. Criterios de selección

A continuación se describen los principales criterios que motivaron la elección de ViT-B/16 frente a otras variantes de *Vision Transformers*:

- Adaptación al dominio visual del dataset: Como se mencionó anteriormente, el dataset objetivo para el entrenamiento pertenece al dominio del e-commerce. Las imágenes presentan productos centrados, fondos neutros y buena calidad visual, condiciones adecuadas para que el mecanismo de atención global de ViT capture patrones discriminativos relevantes. La tarea concreta de clasificación sobre el atributo `articleType`, con más de 100 clases, demanda un modelo con alta capacidad de representación para distinguir diferencias sutiles entre prendas y accesorios.
- Preentrenamiento robusto y transferible: ViT-B/16 ha sido preentrenado sobre ImageNet con una configuración ampliamente validada. Esto permite aprovechar representaciones generales de alto nivel y realizar una adaptación eficiente al dominio específico con un número moderado de epochs y tamaño de batch reducido.
- Compromiso entre capacidad y coste: Frente a variantes más pequeñas (como ViT-Ti o ViT-S), ViT-B ofrece mayor capacidad de representación, lo que resulta ventajoso en un problema con gran número de clases. Al mismo tiempo, evita el alto costo computacional y el riesgo de sobreajuste asociado a variantes más grandes (ViT-L, ViT-H), resultando adecuada para entrenamientos limitados en tiempo y memoria.
- Tamaño de parche equilibrado (16×16): Este tamaño proporciona un balance eficaz entre resolución espacial y eficiencia computacional. Capta suficiente detalle local para discriminar entre productos similares, sin comprometer la viabilidad del entrenamiento con recursos restringidos.
- Explicabilidad del modelo: La arquitectura estándar de ViT-B/16 facilita el análisis e interpretación de las atenciones generadas por el modelo, una característica valiosa en contextos de e-commerce donde se requiere trazabilidad en las decisiones del clasificador.

3.6. Configuración de entrenamiento

Una vez definido el modelo base, se implementó un *pipeline* de entrenamiento en el script `train.py` encargado de orquestrar la construcción de los *DataLoaders*, la inicialización del *Vision Transformer*, el bucle de optimización y el registro de métricas en MLflow. El flujo general es el siguiente:

1. Carga de los archivos `train.csv` y `val.csv` y de las carpetas de imágenes correspondientes.
2. Construcción de los conjuntos de entrenamiento y validación mediante la clase `ImageDataset`, que aplica las transformaciones definidas (redimensionamiento, normalización y, opcionalmente, *data augmentation*).
3. Inicialización del modelo ViT-B/16 preentrenado en ImageNet y adaptación de la *classification head* al número de clases del atributo `articleType`.

4. Entrenamiento del modelo durante un máximo de **10 épocas**, monitorizando la pérdida y las métricas de validación.
5. Aplicación de *early stopping* basado en la pérdida de validación, con una paciencia de **5 épocas** sin mejora.
6. Almacenamiento del mejor modelo en disco y registro de artefactos (modelo y mapeo `class_to_idx`) en MLflow.

3.6.1. Función de pérdida

Para la tarea de clasificación multiclase se utilizó la función de pérdida **CrossEntropyLoss** de PyTorch. Esta elección es estándar en problemas donde cada imagen pertenece exactamente a una única clase entre K posibles, y permite optimizar directamente la probabilidad de la clase correcta a partir de los *logits* producidos por la *classification head*.

Inicialmente se consideró incorporar **ponderación por clase** (*class weighting*) para mitigar el desbalance observado en el atributo `articleType`. Sin embargo, se optó por comenzar con una configuración base sin pesos, utilizando en cambio métricas complementarias (F1 macro y F1 ponderado) para evaluar el impacto del desbalance. La incorporación de *class weighting* se deja identificada como trabajo futuro para profundizar en la corrección de dicho desbalance sin introducir complejidad adicional en esta primera versión del sistema.

3.6.2. Optimizador y regularización

Como algoritmo de optimización se empleó **AdamW** con un *learning rate* inicial de $lr = 3 \times 10^{-4}$ y un término de *weight decay* de 1×10^{-4} . La elección de AdamW se debe a que combina la adaptatividad de Adam (que ajusta de forma independiente el *learning rate* de cada parámetro) con una formulación de *weight decay* desacoplada, lo que ha demostrado funcionar especialmente bien en *Transformers* preentrenados.

El valor de $lr = 3 \times 10^{-4}$ sigue las recomendaciones habituales para *fine-tuning* de modelos preentrenados: lo suficientemente bajo como para no desestabilizar los pesos aprendidos en ImageNet, pero alto como para permitir la adaptación al nuevo dominio con un número limitado de épocas. El término de **weight decay** = 1×10^{-4} introduce una regularización L2 moderada que ayuda a evitar sobreajuste en un problema con más de 100 clases y tamaño de dataset acotado.

3.6.3. Scheduler de *learning rate*

En la configuración actual no se empleó un *scheduler* explícito del *learning rate*. Se optó por mantener un *learning rate* fijo de 3×10^{-4} durante todo el entrenamiento para simplificar el diseño experimental y poder aislar el efecto de otros componentes (data augmentation, *early stopping*, etc.). En trabajos futuros se considera incorporar *learning rate scheduling* (por ejemplo, *cosine decay* o *step decay*) para explorar mejoras adicionales de desempeño una vez fijada la configuración base.

3.6.4. Número de épocas, *batch size* y *early stopping*

El entrenamiento se configuró con un máximo de **10 épocas** y un *batch size* de **32** imágenes. El *batch size* se eligió como compromiso entre el uso eficiente de la

memoria GPU disponible, la estabilidad de las estimaciones de gradiente y un tiempo de entrenamiento razonable para iterar sobre distintos experimentos.

Dado que el modelo parte de pesos preentrenados, no se espera necesitar un número muy elevado de épocas para converger. Para evitar sobreajuste y entrenamientos innecesariamente largos, se implementó un mecanismo de **early stopping** monitorizando la pérdida de validación (`val_loss`) con una paciencia de **5 épocas**. Si la pérdida de validación no mejora en cinco iteraciones consecutivas, el entrenamiento se detiene y se conserva el modelo con mejor desempeño observado.

Este esquema permite adaptarse dinámicamente al punto de saturación del modelo: en experimentos preliminares se observó que la pérdida de validación tiende a estabilizarse antes de alcanzar las 10 épocas, por lo que el *early stopping* ayuda a reducir el tiempo de entrenamiento sin sacrificar rendimiento.

3.6.5. Data augmentation durante el entrenamiento

Durante el entrenamiento se aplicaron técnicas moderadas de *data augmentation* sobre el conjunto de entrenamiento, manteniendo únicamente el redimensionamiento y la normalización en validación. Las transformaciones utilizadas fueron:

- **RandomHorizontalFlip** con probabilidad 0,5.
- **ColorJitter** (brillo, contraste, saturación y tono) aplicado con probabilidad 0,5.
- **RandomRotation** de hasta $\pm 10^\circ$.

Estas transformaciones buscan aumentar la variabilidad efectiva del dataset sin alterar la semántica de la clase (por ejemplo, una remera sigue siendo la misma clase aunque se vea ligeramente más brillante o rotada). La decisión de utilizar augmentations suaves se basó en las observaciones del EDA: fondos mayormente neutros, productos centrados y buena calidad de imagen, donde deformaciones agresivas podrían introducir artefactos irreales.

Capítulo 4

Resultados y Conclusiones

Este capítulo presenta los resultados obtenidos durante el entrenamiento y evaluación del modelo, analiza el desempeño en el conjunto de *test* e interpreta las métricas asociadas. Asimismo, se estudia el comportamiento del modelo durante el entrenamiento, incluyendo la comparación entre distintas configuraciones experimentales y el análisis de mapas de atención. Finalmente, se discuten las principales limitaciones del enfoque y se proponen líneas futuras de trabajo.

4.1. Resultados

La tabla 4.1 resume el desempeño del modelo en el conjunto de *test*. Las métricas reportadas permiten evaluar no sólo el rendimiento global, sino también la estabilidad del comportamiento entre clases, aspecto crítico en contextos donde existen distribuciones desbalanceadas.

TABLA 4.1. Métricas de desempeño del modelo en el conjunto de *test*.

Métrica	Valor
Accuracy	0.8370
F1 Macro	0.5565
F1 Weighted	0.8269
Precision Macro	0.6167
Precision Weighted	0.8317
Recall Macro	0.5467
Recall Weighted	0.8370

El modelo muestra un desempeño global sólido, con un *accuracy* de 0.837. Sin embargo, el valor de *F1 Macro* (0.556) revela que dicho rendimiento no es uniforme entre las distintas clases, afectado principalmente por la baja representación de algunas categorías. Por su parte, las métricas ponderadas (*F1 Weighted*, *Precision Weighted*, *Recall Weighted*) indican un comportamiento muy adecuado para las clases mayoritarias.

En conjunto, estos resultados sugieren que el modelo aprende de manera eficaz para las clases frecuentes, pero presenta limitaciones para aquellas con menor soporte. Técnicas adicionales como aumentación intensiva, *oversampling* o un mayor número de épocas de entrenamiento podrían contribuir a mitigar este efecto.

4.2. Conclusiones

4.2.1. Entrenamiento y validación

Antes de analizar las métricas de *test*, resulta informativo estudiar la evolución del entrenamiento. La figura 4.1 presenta las curvas de pérdida para los tres experimentos considerados, permitiendo observar el comportamiento comparativo entre entrenamientos con y sin aumentación de datos.

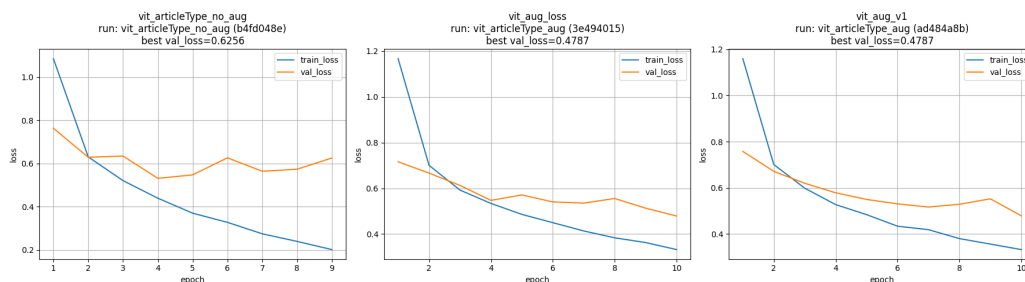


FIGURA 4.1. Evolución de *train loss* y *validation loss* para los distintos experimentos.

El experimento sin aumentación (*vit_articleType_no_aug*) muestra un descenso rápido del *train loss* mientras la pérdida en validación se vuelve inestable, evidenciando sobreajuste temprano. En contraste, las variantes con aumentación presentan curvas más estables y menor pérdida en validación, lo que sugiere mejor capacidad de generalización. Entre ellas, *vit_aug_loss* resulta más consistente al seleccionar el mejor modelo según *val loss*, mientras que *vit_aug_v1* utiliza *accuracy*, un criterio menos adecuado para minimizar la pérdida en validación.

La figura 4.2 muestra de manera directa la comparación entre las pérdidas de validación para los tres entrenamientos.

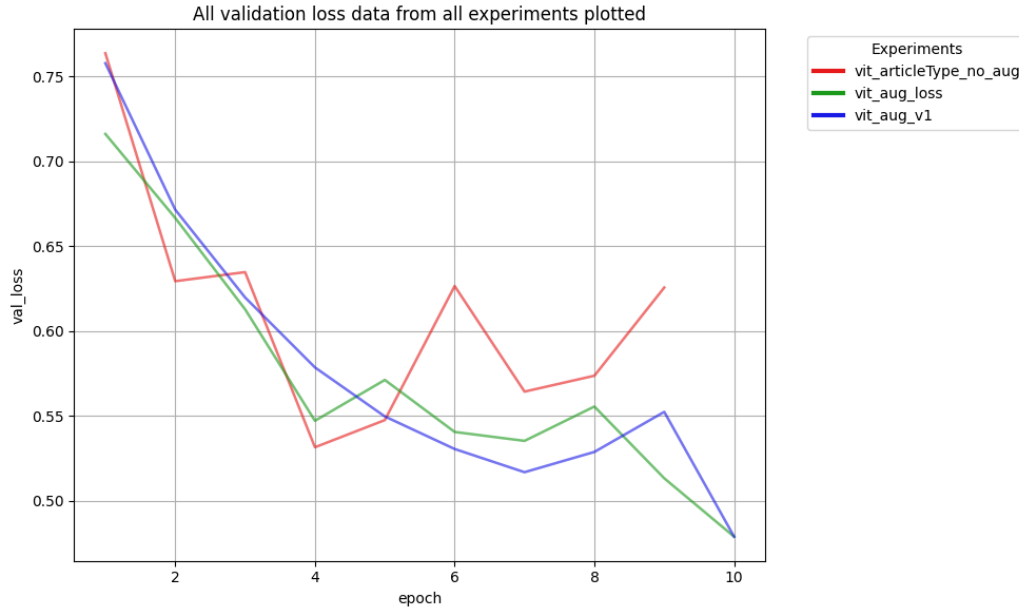


FIGURA 4.2. Comparación del *validation loss* entre los experimentos evaluados.

La comparación confirma que las variantes con aumentación convergen a valores de pérdida más bajos y estables, destacándose nuevamente la configuración `vit_aug_loss`.

4.2.2. Análisis de mapas de atención

La figura 4.3 muestra un mapa de atención generado para una imagen del conjunto de prueba. En este caso, el modelo concentra su activación en los bordes del producto, que contienen información de forma relevante para la clasificación, mientras ignora eficazmente el fondo. Esto sugiere que el mecanismo de atención está capturando patrones significativos y descartando ruido.

Sin embargo, parte de la atención también se distribuye sobre zonas que no aportan información esencial (por ejemplo, partes del cuerpo humano presentes en la imagen). Esto indica que el modelo, si bien aprende a identificar regiones útiles, aún asigna recursos a áreas secundarias, lo que podría introducir cierta ineficiencia o ruido en la representación interna.

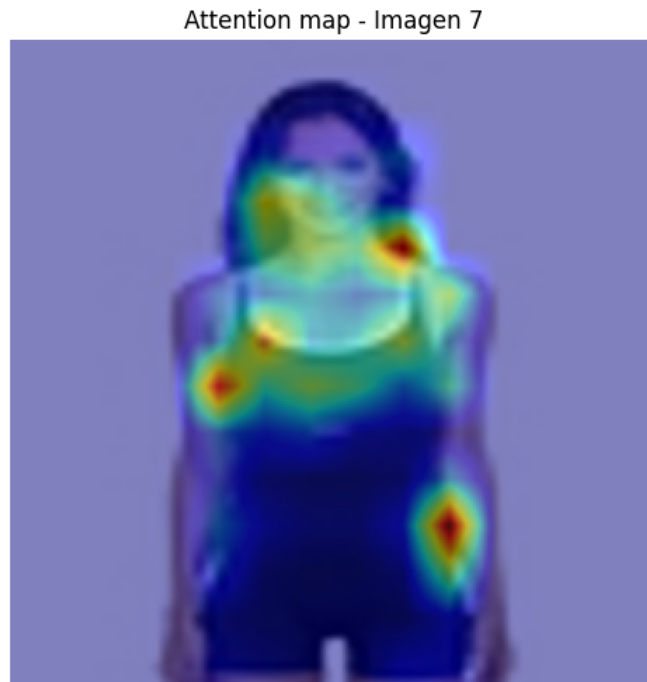


FIGURA 4.3. Ejemplo de mapa de atención generado por el modelo.

4.2.3. Limitaciones y recomendaciones finales

El modelo fue entrenado durante 10 épocas, lo cual es suficiente para observar patrones de comportamiento, pero probablemente insuficiente para alcanzar su máximo potencial. Entrenamientos más largos permitirían evaluar si la arquitectura continúa mejorando o eventualmente sobreajusta.

Por otro lado, sería recomendable comparar este modelo con otras arquitecturas (p. ej., variantes más grandes de Vision Transformers, modelos híbridos o redes convolucionales profundas) para determinar si el desempeño observado es representativo o si otras alternativas podrían superar estos resultados.

Desde una perspectiva de ingeniería de producto, una línea de mejora natural sería la construcción de un *pipeline* de entrenamiento continuo, capaz de incorporar nuevos datos de forma incremental y desplegar actualizaciones del modelo sin intervención manual. Esto permitiría mantener el sistema alineado con cambios en la distribución de los datos o en las categorías del catálogo.

Finalmente, las limitaciones computacionales del entorno local condicionaron el tamaño de los lotes, la duración de los entrenamientos y la exploración de hiperparámetros. El acceso a infraestructura con mayor capacidad de cómputo permitiría realizar experimentos más exhaustivos y evaluar configuraciones más complejas, con potencial para mejorar significativamente el desempeño del modelo.

Apéndice A

Planificación del Proyecto

A continuación se presenta la tabla detallada de planificación del proyecto, que incluye todas las tareas necesarias para completar el trabajo desde la inicialización del repositorio hasta la presentación final.

TABLA A.1. Planificación del Proyecto

Título	Descripción	Responsable	Estado
Inicializar cookiecutter	Ejecutar <code>cookiecutter-data-science</code> para generar la estructura base del proyecto.	Abril	Completado
Configurar entorno Conda	Crear entorno <code>product_tagger</code> e instalar dependencias.	Abril	Completado
Crear repositorio GitHub	Crear repositorio, subir estructura inicial, conectar con VSCode.	Abril	Completado
Definir proyecto final	Acordar que el proyecto será un sistema Product Tagger basado en Vision Transformers.	Equipo	Completado
Definir Business Case	Documentar problema de negocio, costos y beneficios.	Abril	Completado
Obtención de Datos	Implementar descarga automática del dataset desde KaggleHub.	Abril	Completado
EDA	Explorar <code>styles.csv</code> , distribución de atributos y ejemplos de imágenes.	Abril	Completado
Train/Val/Test Split	Crear particiones estratificadas	Abril	Completado
Preparación imágenes	Redimensionar y normalizar imágenes para ViT.	Abril	Completado
Implementar PyTorch Dataset	Construir clase Dataset para imágenes y etiquetas.	Pedro	Completado
Implementar augmentations	Agregar normalización, resize, flips y transforms.	Lautaro	Completado
Cargar Vision Transformer	Cargar ViT/DeiT preentrenado y adaptar classifier.	Pedro	Completado
Pipeline de entrenamiento	Entrenar ViT con optimizer y early stopping.	Lautaro y Pedro	Completado
Hostear MLflow para experimentos	Configurar MLflow para registrar experimentos y métricas del modelo.	Pedro	Completado
Registrar métricas	Guardar loss, accuracy, F1 y mAP por época.	Lautaro	Completado
Evaluar modelo	Evaluar rendimiento sobre test set.	Lautaro y Pedro	Completado
Visualizar resultados	Graficar curvas, métricas y matriz de confusión.	Lautaro	Completado
Ejemplos de predicción	Mostrar aciertos y errores.	Lautaro	Completado
Escribir README	Instrucciones de instalación, entrenamiento e inferencia.	Pedro	Completado
Redactar Objetivo	Escribir objetivo del proyecto en el informe.	Abril	Completado
Redactar Implementación técnica	Detallar módulos y diseño del sistema.	Pedro	Completado
Redactar Evaluación	Explicar métricas y análisis de resultados.	Lautaro	Completado
Redactar Resultados y Conclusiones	Incluir visualizaciones y análisis final.	Lautaro	Completado